

Якунин Владимир 551 группа

ПИ КНИТ (заоч)

НИУ СГУ, Саратов, 2024

Console (выполняется) - Microsoft Visual Studio (Администратор)

Файл Правка Вид Проект Сборка Отладка Команда Средства Тест Анализ Окно Справка

Процесс: [6560] Console.exe

Обозреватель серверов

- Azure (Подключение не установлено)
- Подключения данных
 - user-nk\sqlexpress.TES
- Серверы
 - user-ПК
 - Журналы событий
 - Очереди сообщений
 - Службы
 - Счетчики произво

App.config

Program.cs

```
1 using System;  
2 using System.Collections.Generic;  
3 using System.Linq;
```

Console

Console.UserContext

UserContext()

Средства диагностики

Сеанс диагностики: 2:58 мин

События

Память процесса (МБ)

ЦП (% всех процессоров)

Сводка События Использование памяти Ис

События

Отображение событий (8 из 8)

Исключения (0 из 0)

События IntelliTrace (8 из 8)

Использование памяти

Сделать снимок

Использование ЦП

Профилирование ЦП не поддерживается

Видимые

Имя	Значение	Тип
-----	----------	-----

Видимые

Локальные Контрольные значения 1

Точки останова

Создать

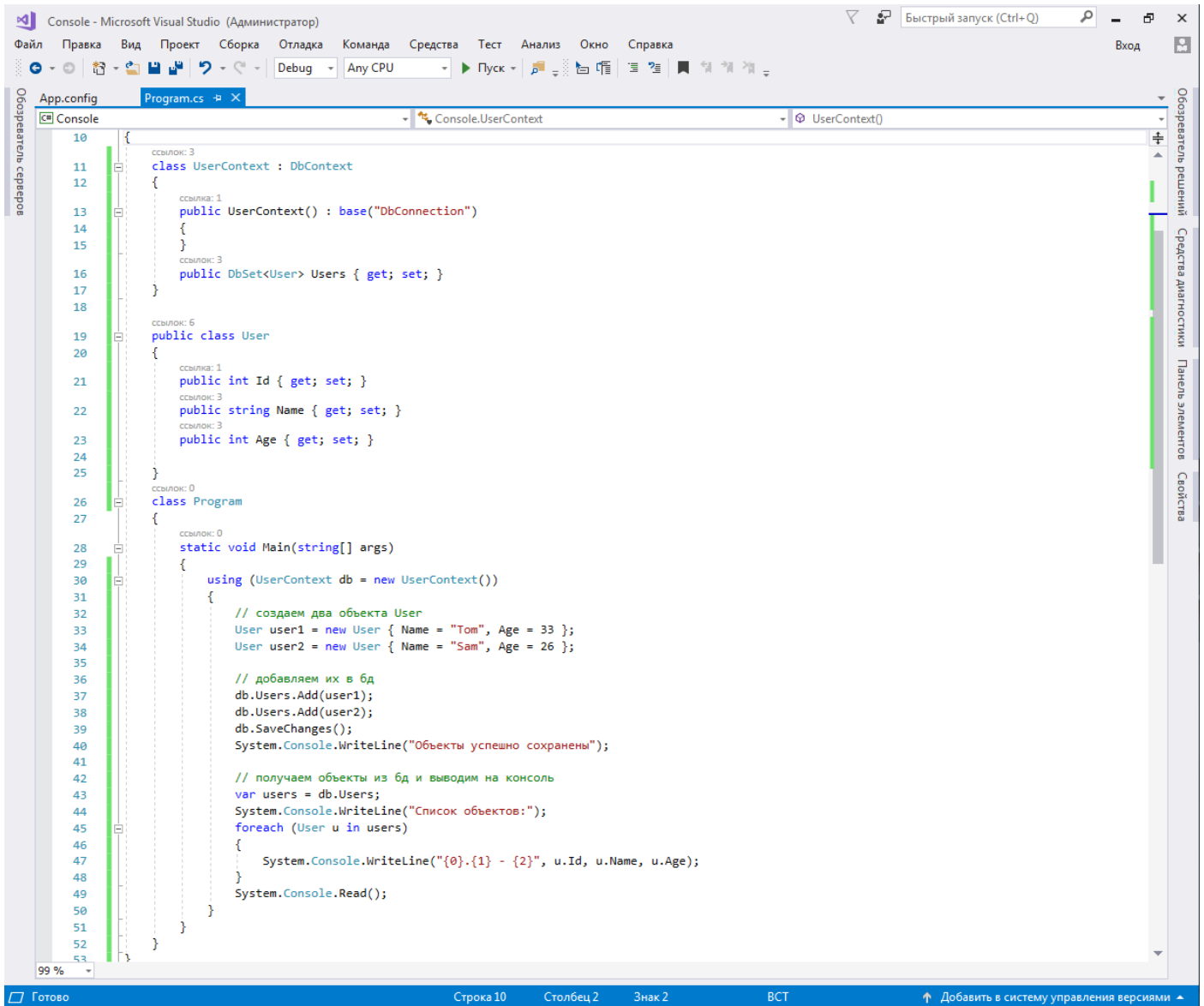
Имя	Метки	Условие	Число попаданий
-----	-------	---------	-----------------

Точки остан...

Параметры... Командное... Окно интер... Вывод

Готово

Добавить в систему управления версиями





1 С помощью какой технологии возможен переход от отношений к объектам? Как она реализована?

Отношения определяют, как две сущности связаны друг с другом. Например, при моделировании сообщений в блоге каждое сообщение связано с блогом, в котором оно опубликовано, а блог связан со всеми сообщениями, опубликованными в этом блоге.

В объектно-ориентированном языке, таком как C#, блог и сообщение обычно представлены двумя классами: `Blog` и `Post`.

В приведенных выше классах нет ничего, что указывало бы на то, что `Blog` и `Post` связаны между собой. Это можно добавить в объектную модель, добавив ссылку `Post` на объект `Blog`, в котором она опубликована.

Аналогично, противоположное направление одной и той же связи можно представить как совокупность `Post`-объектов на каждом из них `Blog`.

Это соединение «от `Blog`» `Post`и, наоборот, «`Post`обратно `Blog`» известно в EF Core как «отношения».

Сопоставление отношений EF Core — это сопоставление представления первичного ключа/внешнего ключа, используемого в реляционной базе данных, со ссылками между объектами, используемыми в объектной модели.

В самом простом смысле это включает в себя:

Добавление свойства первичного ключа к каждому типу сущности.

Добавление свойства внешнего ключа к одному типу сущности.

Связывание ссылок между типами сущностей с первичными и внешними ключами для формирования единой конфигурации отношений.

После создания этого сопоставления EF изменяет значения внешнего ключа по мере необходимости, когда изменяются ссылки между объектами, и изменяет ссылки между объектами по мере необходимости, когда изменяются значения внешнего ключа.

Свойство первичного ключа `Blog`, `Blog.Id` и свойство внешнего ключа `Post`, `Post.BlogId` затем могут быть связаны со ссылками («навигациями») между типами сущностей (`Blog.Posts` и `Post.Blog`). Это делается автоматически EF при построении такой простой связи, но также может быть указано явно при переопределении метода `OnModelCreating` вашего `DbContext`.

2 Какие подходы к разработке клиент-серверных приложений существуют в Entity Framework? В каких случаях какой из них используется?

Как правило, Entity Framework Core пытается выполнить как можно большую часть работы по вычислению запроса на сервере. Некоторые части запроса преобразуются в параметры, которые могут вычисляться на стороне клиента. Остальные части запроса (вместе с созданными параметрами) передаются поставщику базы

данных, который должен определить эквивалентный запрос к базе данных для вычисления на сервере. EF Core поддерживает вычисление на стороне клиента в проекции верхнего уровня (в основном это последний вызов `Select()`). Если проекцию верхнего уровня в запросе невозможно преобразовать на стороне сервера, EF Core получает необходимые данные с сервера и вычисляет оставшиеся части запроса на стороне клиента. Если EF Core обнаруживает выражение в любом месте, кроме проекции верхнего уровня, которую невозможно преобразовать на стороне сервера, то возникает исключение времени выполнения.

Хотя вычисление на стороне клиента полезно, иногда оно может приводить к снижению производительности. Рассмотрим приведенный ниже запрос, в котором вспомогательный метод теперь используется в фильтре `Where`. Так как фильтр невозможно применить в базе данных, все данные необходимо извлечь в память, чтобы применить фильтр в клиенте. В зависимости от фильтра и объема данных на сервере вычисление на стороне клиента может привести к низкой производительности. Поэтому платформа Entity Framework Core блокирует такое вычисление и выдает исключение во время выполнения.

В некоторых случаях, например, в указанных ниже, может потребоваться явное принудительное вычисление на стороне клиента.

Объем данных мал, поэтому вычисление в клиенте не приводит к значительному снижению производительности.

Используемый оператор LINQ не преобразуется на стороне сервера.

В таких случаях можно явно согласиться на вычисление в клиенте, вызвав такой метод, как `AsEnumerable` или `ToList` (`AsAsyncEnumerable` или `ToListAsync` при асинхронном выполнении). При использовании `AsEnumerable` результаты передаются в потоковом режиме, но при использовании `ToList` производится буферизация путем создания списка, который также занимает дополнительную память. Однако если перечисление выполняется несколько раз, сохранять результаты в списке выгоднее, так как запрос к базе данных только один. Вам следует оценить, какой метод полезнее в конкретном случае.

Так как преобразование и компиляция запросов требуют много ресурсов, EF Core кэширует скомпилированный план запроса. Кэшированный делегат может использовать клиентский код при вычислении проекции верхнего уровня на стороне клиента. EF Core создает параметры для вычисляемых клиентом частей дерева и повторно использует план запроса, заменяя значения параметров. Однако некоторые константы в дереве выражения невозможно преобразовать в параметры. Если кэшированный делегат содержит такие константы, эти объекты не могут быть собраны сборщиком мусора, так как на них по-прежнему есть ссылки. Если такой объект содержит `DbContext` или другие службы, это может привести к росту потребления памяти приложением с течением времени. Такое поведение обычно является признаком утечки памяти. EF Core создает исключение всякий раз, когда обнаруживает константы типа, который нельзя сопоставить с помощью текущего поставщика базы данных. Ниже приведены распространенные причины и способы решения.

Использование метода экземпляра: при использовании методов экземпляра в клиентской проекции дерево выражений содержит константу экземпляра. Если метод не использует какие бы то ни было данные из экземпляра, рекомендуется сделать его статическим. Если в теле метода требуются данные экземпляра, передайте их в метод в качестве аргумента.

Передача аргументов константы в метод. Обычно такое происходит при использовании `this` в аргументе для клиентского метода. Рекомендуем разделить аргумент на несколько скалярных аргументов, которые могут быть сопоставлены поставщиком базы данных.

Другие константы: если константа возникает в любом другом случае, можно оценить, требуется ли константа в обработке. Если константа необходима или если ни одно из предыдущих решений не подходит, создайте

локальную переменную для хранения значения и используйте ее в запросе. EF Core преобразует локальную переменную в параметр.

3 Как в модели ней описаны сущности?

В этом разделе описано, как добавлять и присоединять сущности к контексту и как Entity Framework обрабатывает эти сущности во время SaveChanges. Entity Framework заботится о отслеживании состояния сущностей во время подключения к контексту, но в отключенных или N-уровнях сценариев вы можете сообщить EF, в каком состоянии должны находиться ваши сущности. Методы, представленные в этом разделе, также применимы к моделям, созданным с помощью Code First и конструктора EF.

Состояния сущностей и SaveChanges

Сущность может находиться в одном из пяти состояний, определенных перечислением EntityState. Возможны следующие состояния:

Добавлено: сущность отслеживается контекстом, но еще не существует в базе данных

Без изменений: сущность отслеживается контекстом и существует в базе данных, а ее значения свойств не изменились с значений в базе данных.

Изменено: сущность отслеживается контекстом и существует в базе данных, а некоторые или все его значения свойств были изменены.

Удалено: сущность отслеживается контекстом и существует в базе данных, но помечена для удаления из базы данных при следующем вызове SaveChanges

Отсоединение: сущность не отслеживается контекстом

SaveChanges выполняет различные действия для сущностей в разных состояниях:

Неизменяемые сущности не касаются SaveChanges. Обновления не отправляются в базу данных для сущностей в неизменном состоянии.

Добавленные сущности вставляются в базу данных, а затем становятся неизменными при возврате SaveChanges.

Измененные сущности обновляются в базе данных, а затем становятся неизменными при возврате SaveChanges.

Удаленные сущности удаляются из базы данных и затем отсоединяются от контекста.

4 Объясните структуру сгенерированных сущностных классов.

При работе с реляционными базами данных мы используем таблицы со строками и столбцами, такая инфраструктура устройства баз данных является очень структурированной. Перед широким внедрением объектно-ориентированного программирования мы работали в рамках “процедурного” программирования – для решения проблем использовался структурный код и функции. Устройство баз данных в виде таблиц, строк и столбцов было в какой-то степени похоже на структурные и процедурные шаблоны программирования, которые мы использовали в нашем коде. Жизнь была хороша ...

После этого произошла эволюция на стороне кода – сейчас мы уже не используем функции, а думаем в терминах объектов, классов и моделей. Процесс разработки программного обеспечения значительно повзрослел за эти годы и обрел концептуальный уровень – изменилось все, начиная от кода (появились ООП, различные фреймворки и шаблоны облегчающие жизнь) и заканчивая командами разработчиков (современный крупный проект трудно представить без архитекторов, дизайнеров, оптимизаторов и т.д.)

При этом эти изменения практически не затронули базы данных. Данные все также остаются заперты в таблицах, строках и столбцах. Это создает большую пропасть в работе между иерархическим объектно-ориентированным кодом и высоко-нормализованной структурой баз данных. Чтобы справиться с этой проблемой, разработчики предложили внести в программные проекты “объектный слой базы данных”, в котором таблицы баз данных представлялись в виде классов, столбцы в виде свойств, а вся работа с базой данных строилась на программном коде без использования SQL. Такой подход получил название объектно-реляционного отображения (object-relational mapping — ORM).

Различные программные платформы предлагают множество систем, реализующих модель ORM. Entity Framework в сочетании с LINQ (Language-Integrated Query) представляет собой реализацию ORM для платформы .NET Framework от компании Microsoft. Entity Framework содержит механизмы создания и работы с сущностями базы данных через объектно-ориентированный код на языке, совместимым с CLR (в наших примерах мы будем использовать C#). LINQ представляет собой библиотеку, расширяющую возможности C#, и облегчающую создание запросов (благодаря LINQ можно создавать SQL-подобные запросы в коде C#).

Entity Framework является продолжением другого API-интерфейса для работы с базами данных в .NET – ADO.NET, в котором для работы с базами данных приходилось писать запросы на SQL и вставлять их в команды. Думаю как вы поняли, Entity Framework значительно облегчает жизнь программистам C#, слабо разбирающимся в SQL и в деталях работы с базами данных.

История развития Entity Framework

Entity Framework представляет стратегический подход Microsoft к технологии доступа к данным для построения приложений. В отличие от предыдущих технологий доступа к данным, Entity Framework, в сочетании с Visual Studio, обеспечивает широкие возможности работы с базами данных и может использоваться в любых приложениях, будь это веб-сайт, настольное приложение или веб-служба. Давайте кратко рассмотрим хронологию развития Entity Framework.

Впервые Entity Framework появился в 2007 году, а его первая поддержка была обеспечена в Visual Studio 2008, после чего этот API-интерфейс проделал долгий путь и оброс новыми возможностями:

Этапы развития Entity Framework

В первой версии Entity Framework был достаточно ограничен и обеспечивал только базовую реализацию модели ORM на основе подхода Database-First (рассмотрим чуть позже). Версия 4.0 появилась вместе с .NET Framework 4 и Visual Studio 2010 и принесла нам новый подход - Model-First и поддержку POCO-объектов (POCO - Plain Old CLR Object). Вскоре после этого, команда разработчиков Entity Framework выпустила три версии (4.1 – 4.3), в которых поддерживался еще один подход к реализации ORM – Code-First.

Версия 5 Entity Framework вышла вместе с выпуском .NET 4.5 и Visual Studio 2012. Эта версия обеспечивает значительное повышение производительности наряду с поддержкой перечислений, табличных функций, пространственных типов Transact SQL, импорта хранимых процедур, и глубокой интеграцией с ASP.NET MVC.

Текущей версией Entity Framework является 6, в которой появилась поддержка асинхронных запросов и возможность работы с хранимыми процедурами через подход Code-First. Более подробный список изменений в версиях Entity Framework вы можете увидеть на сайте MSDN в статье Entity Framework Version History.

Структура Entity Framework

Теперь давайте кратко рассмотрим некоторые ключевые моменты Entity Framework.

Модель EDM

Entity Framework акцентирует свое внимание на моделировании, в котором вы увидите много знакомых вещей – здесь используются диаграммы ER (entity-relationship, “сущность-отношение”), подход с использованием логического и физического проектирования слоев и многое другое. Ядром Entity Framework является модель EDM (Entity Data Model), суть которой заключается в хранении сущностей (entity) в виде строго типизированных классов, а не в виде объектов схемы базы данных (показано на рисунке ниже). Модель EDM позволяет обеспечить связь между сущностными классами в коде и таблицами базы данных.

Модель EDM

Обратите внимание, что на рисунке выше, таблицы базы данных непосредственно не отображаются на классы сущностей, вместо этого у разработчика есть возможность управлять процессом отображения и он может, например, разбить таблицу на несколько сущностей или наоборот, объединить несколько таблиц в одной сущности (хотя зачастую программисты не используют эту возможность и используют отображение “1 таблица – 1 сущность”).

Слои

Архитектура Entity Framework в абстрактном смысле основана на слоях (layers): рабочий, удаленный и связующий.

Классы с кодом сущностей содержатся в рабочем слое, в котором работают программисты. В зависимости от того, какой подход вы используете (Code-First или DB-First), рабочий слой может быть смоделирован либо с помощью графического дизайнера Visual Studio, либо с помощью кода. После этого у программистов появляется широкий инструментарий для работы с Entity Framework. Синтаксис рабочего слоя описывается с помощью языка Conceptual Schema Definition Language (CSDL).

Удаленный слой определяет таблицы, столбцы, строки, отношения между таблицами базы данных. Синтаксис удаленного слоя описывается с помощью языка Store Schema Definition Language (SSDL).

Связующий слой определяет соответствие между рабочим и удаленным слоями, он связывает свойства сущностного класса в рабочем слое со столбцами таблицы базы данных в удаленном слое. Управлять этим слоем (т.е. деталями привязки) можно из окна Mapping Details находящегося в инструментах дизайнера Visual Studio или с помощью аннотаций Fluent API, если вы работаете с подходом Code-First. Язык Mapping Specification Language (MSL) определяет синтаксис связующего слоя.

Важно отметить, что языки CSDL, SSDL и MSL имеют синтаксис XML, но при этом используют разную семантику.

Файлы Entity Framework

Все файлы, используемые в Entity Framework основаны на синтаксисе XML. Использование XML делает файлы простыми и универсальными для других приложений. Также XML-файлы читабельны для человека – вы можете в любой момент открыть и просмотреть содержимое этих файлов. Тем не менее, каждый элемент Entity Framework использует различные файлы XML с различным расширением.

После того как вы создадите новое приложение, которое опирается на Entity Framework и добавите сущностные классы базы данных, вы сможете увидеть результирующие файлы в основной папке проекта (в следующей статье мы более подробно опишем генерацию сущностных классов). В среде Visual Studio 2012 вы найдете единственный файл Entity Data Model XML (.EDMX), хотя в более старых версиях Visual Studio возможно будет создано несколько файлов (один для каждой сущности).

Файл EDMX

Файл EDMX содержит в себе несколько секций, написанных на языках CSDL, SSDL и MSL, представляющих разные слои в архитектуре Entity Framework. Чтобы открыть исходный код этого файла в Visual Studio, щелкните по нему правой кнопкой мыши и выберите в контекстном меню Open With..., после чего в диалоговом окне выберите вариант XML Editor.

Раздел CSDL содержит XML-код, необходимый для построения рабочего слоя:

```
<!-- CSDL content -->
```

```
<edmx:ConceptualModels>
```

```
  <Schema Namespace="SampleDbModel" ...>
```

```
    <EntityType Name="Employee">
```

```
      <Key>
```

```
        <PropertyRef Name="Id" />
```

```
      </Key>
```

```
      <Property Name="Id" Type="Int32" Nullable="false" />
```

```
      <Property Name="FirstName" Type="String" MaxLength="20"
```

```
        FixedLength="true" Unicode="true" Nullable="false" />
```

```
      <Property Name="LastName" Type="String" MaxLength="20"
```

```
        FixedLength="true" Unicode="true" Nullable="false" />
```

```
      <Property Name="DepartamentNumber" Type="String" MaxLength="4"
```

```
        FixedLength="true" Unicode="false" />
```

```
      <NavigationProperty Name="Department"
```

```
        Relationship="Self.FK_Employee_Department"
```

```
        FromRole="Employee" ToRole="Department" />
```

```
      <NavigationProperty Name="Works_on"
```

```
        Relationship="Self.FK_Works_on_Employee"
```

```

        FromRole="Employee" ToRole="Works_on" />
    </EntityType>
    <EntityContainer Name="SampleDbEntities" annotation:LazyLoadingEnabled="true">
        <EntitySet Name="Department" EntityType="Self.Department" />
        <EntitySet Name="Employee" EntityType="Self.Employee" />
        <EntitySet Name="Project" EntityType="Self.Project" />
        <EntitySet Name="sysdiagrams" EntityType="Self.sysdiagrams" />
        <EntitySet Name="Works_on" EntityType="Self.Works_on" />
        <AssociationSet Name="FK_Employee_Department" Association="Self.FK_Employee_Department">
            <End Role="Department" EntitySet="Department" />
            <End Role="Employee" EntitySet="Employee" />
        </AssociationSet>
        ...
    </EntityContainer>
</Schema>
</edmx:ConceptualModels>

```

Обратите внимание, что в CSDL описываются сгенерированные сущности и связи между ними в разделе EntityContainer. В элементах EntityType описывается структура сущностей – имена столбцов и первичные ключи. Обратите внимание на атрибут Type, в котором указывается тип сущности – в данном случае используются .NET-совместимые типы, а не типы T-SQL.

Секция с кодом SSDL описывает структуру таблиц базы данных, их структуру, хранимые процедуры и т.д. Разделы EntityContainer и EntityType аналогичны секции CSDL, но записи опираются на типы данных T-SQL, а не .NET. В примере ниже показана часть секции с кодом SSDL:

```

<!-- SSDL content -->
<edmx:StorageModels>
    <Schema Namespace="SampleDbModel.Store" ...>
        <EntityType Name="Employee">
            <Key>
                <PropertyRef Name="Id" />
            </Key>
            <Property Name="Id" Type="int" Nullable="false" />
            <Property Name="FirstName" Type="nvarchar" MaxLength="20" Nullable="false" />

```

```

    <Property Name="LastName" Type="nchar" MaxLength="20" Nullable="false" />

    <Property Name="DepartamentNumber" Type="char" MaxLength="4" />

</EntityType>

...

<EntityContainer Name="SampleDbModelStoreContainer">

    <EntitySet Name="Employee" EntityType="Self.Employee" Schema="dbo" store:Type="Tables" />

    ...

    <AssociationSet Name="FK_Employee_Department" Association="Self.FK_Employee_Department">

        <End Role="Department" EntitySet="Department" />

        <End Role="Employee" EntitySet="Employee" />

    </AssociationSet>

    ...

</EntityContainer>

</Schema>

</edmx:StorageModels>

```

Код MDL связывает сущности, указанные в разделах SSDL и CSDL, и определяет как будут отображаться данные из базы данных на классы .NET. В примере ниже показана часть секции с кодом MDL:

```

<!-- C-S mapping content -->

<edmx:Mappings>

    <Mapping Space="C-S" xmlns="http://schemas.microsoft.com/ado/2009/11/mapping/cs">

        <EntityContainerMapping ...>

            <EntitySetMapping Name="Employee">

                <EntityTypeMapping TypeName="SampleDbModel.Employee">

                    <MappingFragment StoreEntitySet="Employee">

                        <ScalarProperty Name="Id" ColumnName="Id" />

                        <ScalarProperty Name="FirstName" ColumnName="FirstName" />

                        <ScalarProperty Name="LastName" ColumnName="LastName" />

                        <ScalarProperty Name="DepartamentNumber" ColumnName="DepartamentNumber" />

                    </MappingFragment>

                </EntityTypeMapping>

            </EntitySetMapping>

        </Mapping>

    </edmx:Mappings>

```

...

</EntityContainerMapping>

</Mapping>

</edmx:Mappings>

Подходы для работы с Entity Framework

В Entity Framework поддерживается три подхода к разработке:

Database-First

Подходит для проектировщиков баз данных - сначала вы создаете базу данных с помощью различных инструментов (например, SQL Server Management Studio), а затем генерируете EDMX-модель базы данных, которая предоставляет удобный графический интерфейс для взаимодействия с базой данных в виде диаграмм и объектную модель в виде классов C#. В данном случае вам нужно работать с SQL Server и хорошо знать синтаксис T-SQL, но при этом не нужно разбираться в C#.

Model-First

Подходит для архитекторов - сначала вы создаете графическую модель EDMX в Visual Studio (в фоновом режиме создаются классы C# модели), а затем генерируете на основе диаграммы EDMX базу данных. При данном подходе не нужно знать ни деталей T-SQL ни синтаксиса C#.

Code-First

Подходит для программистов - при данном подходе модель EDMX вообще не используется и вы вручную настраиваете классы C# объектной модели (данный подход поддерживает как генерацию сущностных классов из существующей базы данных, так и создание базы данных из созданной вручную модели объектов C#). Очевидно, что это подходит для программистов, хорошо знакомых с синтаксисом C#.